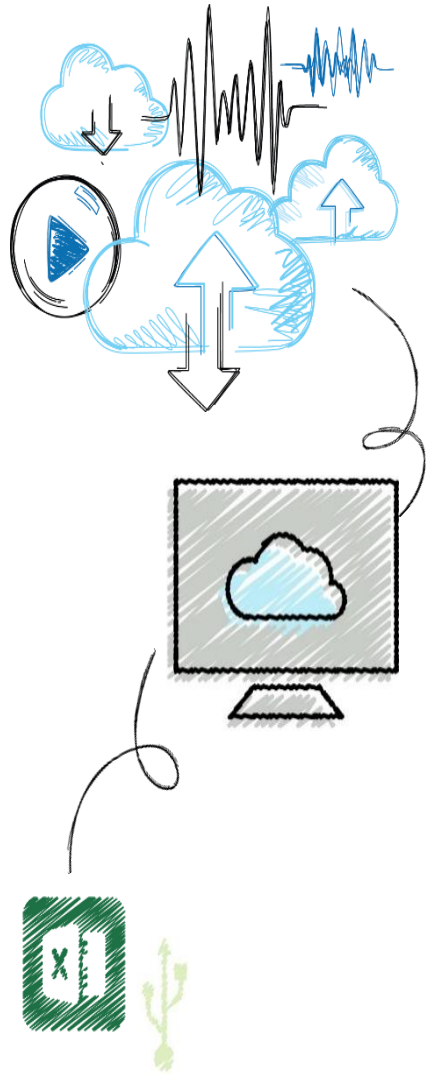


ICT703 Programming

Module 6

Dictionaries





Objectives

1. Construct dictionaries and access entries in those dictionaries
2. Use methods to manipulate dictionaries
3. Determine whether a list or a dictionary is an appropriate data structure for a given application



Programming Concepts

Four control flow

- Sequence
- Iteration
- Selection
- Abstraction – methods/classes

Data



Introduction

- A **list** allows the programmer to manipulate a sequence of data values of any types
- A **dictionary** organizes data values by association with other data values rather than by sequential position
- Lists and dictionaries provide powerful ways to organize data in useful and interesting applications



Dictionaries

- A dictionary organizes information by **association**, not position
 - Example: When you use a dictionary to look up the definition of “mammal,” you don’t start at page 1; instead, you turn to the words beginning with “M”
- Data structures organized by association are also called **tables** or **association lists**
- In Python, a **dictionary** associates a set of **keys** with data values



Dictionary Literals

- A Python dictionary is written as a sequence of key/value pairs separated by commas
 - Pairs are sometimes called **entries**
 - Enclosed in curly braces { and }
 - A colon (:) separates a key and its value
 - Examples:
 - A phone book: **{‘Savannah’:‘8476-3321’, ‘Nathaniel’:‘8351-7743’}**
 - Personal information: **{‘Name’:‘Molly’, ‘Age’:18}**
 - An empty dictionary: **{}**
 - Keys in a dictionary can be data of any immutable types, including other data structures
 - They are normally strings or integers
-



Adding Keys and Replacing Values

- Add a new key/value pair to a dictionary using []:

<a dictionary>[<a key>] = <a value>

- Example:

```
info = {}  
info["name"] = "Sandy"  
info["occupation"] = "hacker"  
print(info)  
{'name': 'Sandy', 'occupation': 'hacker'}
```

- Use [] also to replace a value at an existing key:

```
info["occupation"] = "manager"  
print(info)  
{'name': 'Sandy', 'occupation': 'manager'}
```



Accessing Values (1 of 2)

- Use `[]` to obtain the value associated with a key
- If key is not present in dictionary, an error is raised

```
print(info["name"])  
'Sandy'
```

```
print( info["job"])
```

Traceback (most recent call last):

File "<pyshell#1>", line 1, in <module>

info["job"]

KeyError: 'job'



Accessing Values (2 of 2)

- If the existence of a key is uncertain, test for it using the dictionary method **has_key**
 - Easier strategy is to use the method **get**

if "job" in info:

 print(info["job"])



Removing Keys

- To delete an entry from a dictionary, remove its key using the method **pop**
 - **pop** expects a key and an optional default value as arguments

```
print(info.pop("job", None))
```

```
None
```

```
print(info.pop("occupation"))
```

```
Manager
```

```
print(info)
```

```
{'name': 'Sandy'}
```



Traversing a Dictionary (1 of 3)

- To print all of the keys and their values:

```
for key in info:  
    print(key, info[key])
```

- Alternative: Use the dictionary method **items()**

```
grades = {90:'A', 80:'B', 70:'C'}
```

```
print(list(grades.items()))  
[(80, 'B'), (90, 'A'), (70, 'C')]
```

- Entries are represented as tuples within the list

```
for (key, value) in grades.items():  
    print(key, value)
```



Traversing a Dictionary (2 of 3)

- You can sort the list first then traverse it to print the entries of the dictionary in alphabetical order:

```
theKeys = list(info.keys())  
theKeys.sort()  
for key in theKeys:  
    print(key, info[key])
```



Traversing a Dictionary (3 of 3)

Dictionary Operation	What It Does
<code>len(d)</code>	Returns the number of entries in d
<code>d[key]</code>	Used for inserting a new key, replacing a value, or obtaining a value at an existing key
<code>d.get(key [, default])</code>	Returns the value if the key exists or returns the default if the key does not exist
<code>d.pop(key [, default])</code>	Removes the key and returns the value if the key exists or returns the default if the key does not exist
<code>list(d.keys())</code>	Returns a list of the keys
<code>list(d.values())</code>	Returns a list of the values
<code>list(d.items())</code>	Returns a list of tuples containing the keys and values for each entry
<code>d.clear()</code>	Removes all the keys
<code>for key in d:</code>	key is bound to each key in d in an unspecified order



Example: The Hexadecimal System

- You can keep a hex-to-binary **lookup dictionary** to aid in the conversion process

```
hexToBinary = {'0':'0000', '1':'0001', '2':'0010',  
'3':'0011', '4':'0100', '5':'0101',  
'6':'0110', '7':'0111', '8':'1000',  
'9':'1001', 'A':'1010', 'B':'1011',  
'C':'1100', 'D':'1101', 'E':'1110',  
'F':'1111'}
```



Example: The Hexadecimal System

- Use it to convert the number
- Example:

binary=""

number="35A"

for digit in number:

binary = hexToBinary[digit] + binary



Example: Finding the Mode of a List of Values

- Find the most frequent word in a file

Get the file

Convert its words to uppercase (or lowercase) so all words are the same case

Add the words to a list

Save each unique word and their frequencies in a dictionary

When the word is first seen it should be a value of 1

If it is not first seen, then it should be incremented by 1

Get the maximum value in the dictionary and print the word

```
fileName = "essay.txt"  
f = open(fileName, 'r' )
```

```
# Input the text, convert its words to uppercase, and add the words to a list  
words = []  
for line in f:  
    for word in line.split():  
        words.append(word.upper())
```

```
# Obtain the set of unique words and their frequencies, saving them in a dictionary  
theDictionary = {}  
for word in words:  
    number = theDictionary.get(word, None)  
    if number == None:  
        # word entered for the first time  
        theDictionary[word] = 1  
    else:  
        # word already seen, increment its number  
        theDictionary[word] = number+1
```

```
# Find the mode by obtaining the maximum value in the dictionary and determining its key  
theMaximum = max(theDictionary.values())  
for key in theDictionary:  
    if theDictionary[key] == theMaximum:  
        print("The mode is", key)  
        break
```



Summary

- A dictionary associates a set of keys with values
 - `[]` is used to add a new key/value pair to a dictionary or to replace a value associated with an existing key
 - **dict** type includes methods to access and remove data in a dictionary